

# PyTorch Einführungs-Skript für Iris

*Ein Einführungsskript zur Klassifikation des Iris-Datensatzes mit PyTorch, inklusive Varianten für Batching, Early Stopping, Skalierung, Visualisierung und Learning Rate Scheduling.*

## Einstieg mit dem einfachst denkbaren Trainings-Skript

```

import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

# Laden des Iris-Datensatzes
iris = load_iris()
X = iris.data
y = iris.target

# Aufteilen des Datensatzes in Trainings- und Testdaten
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2, random_state=0)

# Umwandeln der Daten in PyTorch-Tensoren
X_train = torch.from_numpy(X_train).float()
X_test = torch.from_numpy(X_test).float()
y_train = torch.from_numpy(y_train).long()
y_test = torch.from_numpy(y_test).long()
X_val = torch.from_numpy(X_val).float()
y_val = torch.from_numpy(y_val).long()

# Definieren des neuronalen Netzes als Sequential-Modell
net = nn.Sequential(
    nn.Linear(4, 10),
    nn.ReLU(),
    nn.Linear(10, 3)
)

# Definieren des Verlustkriteriums und des Optimierers
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(net.parameters(), lr=0.01)

# Trainieren des neuronalen Netzes
for epoch in range(100):
    net.train()
    optimizer.zero_grad()
    outputs = net(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()
    # Validierung nach jeder Epoche
    net.eval()
    with torch.no_grad():
        val_out = net(X_val)
        val_loss = criterion(val_out, y_val)

# Auswerten des neuronalen Netzes auf den Testdaten
with torch.no_grad():
    outputs = net(X_test)
    _, predicted = torch.max(outputs, 1)
    accuracy = accuracy_score(y_test, predicted)
    print('Testgenauigkeit: ', accuracy)

# Abspeichern des trainierten Netzes
torch.save(net.state_dict(), 'iris_net.pth')
print("Trainiertes Netz abgespeichert als 'iris_net.pth'")

```

## Aufbau des Skripts

**Importieren der Bibliotheken:** Wir importieren die notwendigen Bibliotheken: torch für PyTorch, torch.nn für neuronale Netze, torch.optim für Optimierungsalgorithmen, sklearn.datasets für den Iris-Datensatz und sklearn.model\_selection für die Aufteilung des Datensatzes.

**Laden des Iris-Datensatzes:** Wir laden den Iris-Datensatz mithilfe von load\_iris() aus sklearn.datasets.

**Aufteilen des Datensatzes:** Wir teilen den Datensatz in Trainings- und Testdaten auf mithilfe von train\_test\_split() aus sklearn.model\_selection.

**Umwandeln der Daten in PyTorch-Tensoren:** Wir wandeln die Daten in PyTorch-Tensoren um, indem wir torch.from\_numpy() verwenden. Dies teilt den Speicher mit dem NumPy-Array und ist effizienter als torch.tensor().

**Definieren des neuronalen Netzes:** Wir definieren ein einfaches neuronales Netz mit nn.Sequential: einer Eingabeschicht mit 4 Merkmalen, einer versteckten Schicht mit 10 Neuronen und einer Ausgabeschicht mit 3 Klassen.

**Instanzieren des Optimierers:** Wir instanzieren den Optimierer (in diesem Fall AdamW mit einer Lernrate von 0,01).

**Trainieren des neuronalen Netzes:** Wir trainieren das neuronale Netz für 100 Epochen. In jeder Epoche berechnen wir die Ausgabe des Netzes, den Verlust und aktualisieren die Parameter mithilfe des Optimierers.

**Auswerten auf den Testdaten:** Wir werten das neuronale Netz auf den Testdaten aus, indem wir die Ausgabe berechnen und die Genauigkeit bestimmen.

## Erläuterungen

### `net.eval()`

Schaltet das Netz in den Auswertungsmodus, d. h. es werden bestimmte Operationen deaktiviert oder anders ausgeführt, um die Auswertung zu optimieren. Insbesondere werden:

**Dropout-Schichten deaktiviert:** Dropout ist eine Technik, die während des Trainings verwendet wird, um Überanpassung zu vermeiden. Im Auswertungsmodus werden diese Schichten deaktiviert, um die Ausgabe des Netzes zu stabilisieren.

**Batch-Normalisierung verwendet laufende Statistiken:** Batch-Normalisierung normalisiert die Eingaben während des Trainings mithilfe der Statistiken der aktuellen Batch. Im Auswertungsmodus werden stattdessen die laufenden Statistiken verwendet, die während des Trainings gesammelt wurden.

`net.eval()` ist wichtig, weil es sicherstellt, dass das neuronale Netzwerk während der Auswertung korrekt funktioniert. Wenn ein Netz im Trainingsmodus ist, kann es zu unerwartetem Verhalten kommen, wie z. B. zufälligen Ausgaben aufgrund von Dropout-Schichten.

### `with torch.no_grad():`

Dieser Kontextmanager deaktiviert die Berechnung von Gradienten für die darin enthaltenen Operationen. Wenn wir das neuronale Netz auswerten (z. B. auf den Testdaten), benötigen wir keine Gradienten, da wir die Parameter nicht aktualisieren.

### `optimizer.zero_grad()`

Setzt die Gradienten der Parameter des neuronalen Netzes auf Null. Wenn wir das neuronale Netz trainieren, berechnen wir den Verlust (Loss) zwischen den vorhergesagten und den tatsächlichen Werten. Anschließend berechnen wir die Gradienten der Parameter mithilfe des Backpropagation-Algorithmus. Diese Gradienten geben an, wie stark jeder Parameter zum Verlust beiträgt. Der Optimierer verwendet diese Gradienten, um die Parameter zu aktualisieren.

Wenn wir jedoch in der nächsten Iteration wieder den Verlust und die Gradienten berechnen, würden wir die Gradienten der vorherigen Iteration addieren. Um dies zu vermeiden, setzen wir die Gradienten auf Null, bevor wir den Verlust berechnen. Dies stellt sicher, dass die Gradienten korrekt sind und der Optimierer die Parameter korrekt aktualisiert.

## Wichtige Hyperparameter für MLP-Setups

| Hyperparameter           | Beschreibung / Relevanz                                                                               | Typische Werte                       |
|--------------------------|-------------------------------------------------------------------------------------------------------|--------------------------------------|
| Anzahl der Hidden-Layer  | Mehr Schichten können komplexere Zusammenhänge abbilden, erhöhen aber Trainingszeit und Overfitting.  | 1–3                                  |
| Lernrate (Learning Rate) | Einfluss darauf, wie stark sich Gewichte bei jedem Schritt verändern.                                 | 0.001–0.01 (Adam), 0.01–0.1 (SGD)    |
| Aktivierungsfunktion     | Beeinflusst die Lernfähigkeit und Stabilität des Modells.                                             | ReLU, Leaky ReLU, ELU, Tanh (selten) |
| Batch Size               | Anzahl der Datensätze, die gleichzeitig verarbeitet werden; beeinflusst Konvergenz und Trainingszeit. | 32, 64, 128, 256                     |
| Dropout-Rate             | Reguliert die Generalisierung, verhindert Überanpassung.                                              | 0.2 bis 0.5                          |
| Optimizer                | Algorithmus zur Anpassung der Netzwerkgewichte.                                                       | Adam, RMSprop, SGD (+Momentum)       |
| Gewichtsinitialisierung  | Art der Initialisierung beeinflusst das Lernverhalten.                                                | Xavier (Glorot), He                  |
| Regularisierung (L1, L2) | Weitere Regularisierungsoptionen, die Gewichte begrenzen, um Overfitting zu reduzieren.               | L2: 0.0001–0.001                     |
| Epochs                   | Anzahl der Durchläufe über den Trainingsdatensatz.                                                    | Typisch: 20–50; siehe Early Stopping |
| Early Stopping           | Beendet das Training, wenn die Performance auf Validierungsdaten stagniert.                           | Patience: 3–10 Epochs                |

Die Wahl der Lernrate und des Optimizers **hat oft den größten Einfluss** auf das Modellverhalten. Dropout und Regularisierung sind zentrale Mittel zur Sicherstellung der Generalisierungsfähigkeit. Aktivierungsfunktionen wie ReLU haben sich empirisch bewährt und sollten standardmäßig verwendet werden. Early Stopping sollte eingesetzt werden, um unnötig langes Training und Overfitting zu verhindern.

## Warum kein Softmax im Output-Layer?

In PyTorch wird bei Klassifikationsproblemen mit `nn.CrossEntropyLoss()` im Hintergrund bereits Folgendes durchgeführt:

**Logits statt Wahrscheinlichkeiten:** Das Modell liefert mit `nn.Linear(10, 3)` drei rohe Scores (Logits) für jede der drei Klassen. Diese Logits enthalten noch keine Softmax-Normalisierung.

**Softmax + Log + NLL in der Loss-Funktion:** `nn.CrossEntropyLoss()` kombiniert intern `nn.LogSoftmax(dim=1)` und `nn.NLLLoss()`:

```
log_probs = F.log_softmax(logits, dim=1)
loss = NLLLoss(log_probs, targets)
```

Deshalb muss im Modell nicht noch explizit eine Softmax-Schicht hinzugefügt werden, wenn direkt mit `CrossEntropyLoss` trainiert wird.

## Optimizer-Settings

| Bezeichnung       | Funktionsweise                                                                                                                               | Syntax                                                                                           |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| SGD               | Aktualisiert die Parameter anhand des negativen Gradienten der Verlustfunktion, skaliert durch die Lernrate.                                 | <code>optim.SGD(params, lr=0.01)</code><br><code>optim.SGD(params, lr=0.01, momentum=0.9)</code> |
| Adagrad           | Passt die Lernrate für jeden Parameter individuell an, basierend auf der Häufigkeit von Updates. Lernrate sinkt mit der Zeit.                | <code>optim.Adagrad(params, lr=0.01)</code>                                                      |
| <i>Adadelat</i>   | <i>Eine Erweiterung von Adagrad, die die Lernrate adaptiv anpasst, ohne eine globale Lernrate zu benötigen.</i>                              | <code>optim.Adadelat(params, lr)</code>                                                          |
| <i>RMSprop</i>    | <i>Teilt die Lernrate durch einen exponentiell gewichteten Durchschnitt der Quadrate der Gradienten.</i>                                     | <code>optim.RMSprop(params, lr)</code>                                                           |
| Adam              | Kombiniert die Vorteile von Adagrad und RMSprop, indem die Lernrate adaptiv angepasst wird. Speichert Momentumschätzungen 1. und 2. Ordnung. | <code>optim.Adam(params, lr=0.001)</code>                                                        |
| AdamW             | Eine Variante von Adam, die die Gewichtsdekadenz korrigiert.                                                                                 | <code>optim.AdamW(params, lr=0.0001)</code>                                                      |
| <i>SparseAdam</i> | <i>Eine Variante von Adam, optimiert für dünn besetzte Tensoren.</i>                                                                         | <code>optim.SparseAdam(params, lr)</code>                                                        |
| <i>LBFGS</i>      | <i>Ein Quasi-Newton-Verfahren, das die Hesse-Matrix approximiert.</i>                                                                        | <code>optim.LBFGS(params, lr)</code>                                                             |
| <i>NAdam</i>      | <i>Variante von Adam mit Nesterov-Beschleunigung.</i>                                                                                        | <code>optim.NAdam(params, lr)</code>                                                             |
| <i>RAdam</i>      | <i>Variante von Adam, die die Varianz der Gradienten reduziert.</i>                                                                          | <code>optim.RAdam(params, lr)</code>                                                             |
| <i>Adamax</i>     | <i>Variante von Adam, die die Maximum-Norm verwendet.</i>                                                                                    | <code>optim.Adamax(params, lr)</code>                                                            |

## Criterion-Settings (Loss-Funktionen)

| Bezeichnung               | Anwendung             | Funktionsweise                                                                                    | Implementierung                       |
|---------------------------|-----------------------|---------------------------------------------------------------------------------------------------|---------------------------------------|
| Mean Squared Error (MSE)  | Regression            | Berechnet den durchschnittlichen quadratischen Fehler zwischen Vorhersage und tatsächlichem Wert. | <code>nn.MSELoss()</code>             |
| Mean Absolute Error (MAE) | Regression            | Berechnet den durchschnittlichen absoluten Fehler zwischen Vorhersage und tatsächlichem Wert.     | <code>nn.L1Loss()</code>              |
| Binary Cross-Entropy      | Binäre Klassifikation | Berechnet den Fehler bei binären Klassifikationsproblemen.                                        | <code>nn.BCELoss()</code>             |
| KL Divergence             | Verteilungsvergleich  | Misst die Differenz zwischen zwei Wahrscheinlichkeitsverteilungen.                                | <code>nn.KLDivLoss()</code>           |
| Cosine Embedding          | Ähnlichkeitsmessung   | Misst die Ähnlichkeit zwischen zwei Vektoren.                                                     | <code>nn.CosineEmbeddingLoss()</code> |
| Hinge Embedding           | Binäre Klassifikation | Berechnet den Fehler bei binären Klassifikationsproblemen mit einer bestimmten Margin.            | <code>nn.HingeEmbeddingLoss()</code>  |
| Margin Ranking            | Ranking               | Berechnet den Fehler zwischen zwei Vorhersagen und einem tatsächlichen Ranking.                   | <code>nn.MarginRankingLoss()</code>   |
| Triplet Margin            | Metrisches Lernen     | Berechnet den Fehler zwischen drei Vorhersagen (positiv, negativ, anchor).                        | <code>nn.TripletMarginLoss()</code>   |

Es gibt viele weitere Loss-Funktionen in PyTorch, aber diese Liste enthält einige der gängigsten und wichtigsten. Die Wahl der Loss-Funktion hängt vom spezifischen Problem und der Aufgabe ab.

## Wie viele Neuronen im Hidden-Layer: Faustregeln

**Zwischen Anzahl der Eingaben und Ausgaben:** Die Anzahl der Neuronen im ersten Hidden Layer sollte zwischen der Anzahl der Eingangs- und Ausgangsneuronen liegen. Beispiel: 10 Input-Features und 2 Ausgaben → Start mit 6 Neuronen im Hidden Layer.

**2/3-Regel (Heuristik):** Wähle die Anzahl der Neuronen im ersten Hidden Layer als etwa zwei Drittel der Eingabeneuronen. Diese Regel stammt aus früheren ANN-Zeiten und ist nur ein Startpunkt.

**Anzahl der Neuronen = Anzahl der Input-Features  $\pm$  k:** Dabei ist k eine kleine Zahl. Beispiel: Input = 20 → Hidden Layer: 15–25 Neuronen.

## Begründungen und Überlegungen

**Overfitting vs. Underfitting:** Zu viele Neuronen → Gefahr von Overfitting (Netz merkt sich Training zu gut). Zu wenige Neuronen → Gefahr von Underfitting (Netz kann Muster nicht erkennen).

**Komplexität der Aufgabe:** Einfache Aufgaben (z. B. lineare Trennung): wenige Neuronen. Komplexe Aufgaben (z. B. Bilderkennung): mehr Neuronen nötig.

**Datensatzgröße:** Wenig Trainingsdaten → weniger Neuronen (Overfitting-Gefahr!). Viele Trainingsdaten → mehr Neuronen möglich.

**Experimentell optimierbar:** In der Praxis wird die ideale Anzahl per Cross-Validation oder durch systematisches Testen (z. B. Grid Search) gefunden.



## Universelles Approximationstheorem

Das Theorem (z. B. nach Cybenko, 1989 oder Hornik, 1991) besagt:

**Ein neuronales Netzwerk mit einem einzigen Hidden Layer und ausreichend vielen Neuronen kann jede stetige Funktion** auf einem kompakten Intervall **beliebig genau approximieren**, sofern eine nichtlineare Aktivierungsfunktion (z. B. Sigmoid oder ReLU) verwendet wird.

### Zusammenhang zur Anzahl der Neuronen

Das Theorem garantiert Existenz, aber nicht Effizienz: Ja, ein einziger Hidden Layer reicht theoretisch – wenn genug Neuronen da sind. In der Praxis braucht man oft viele Neuronen, um komplexe Funktionen abzubilden.

**Zu wenige Neuronen** → Netz kann die Funktion nicht gut approximieren (Underfitting). **Mehr Neuronen** → bessere Approximation, aber steigende Gefahr von Overfitting.

| Annahme                            | Bedeutung                                                  |
|------------------------------------|------------------------------------------------------------|
| 1 Hidden Layer                     | Reicht theoretisch.                                        |
| "Genug Neuronen"                   | Nicht spezifiziert → abhängig von Problemkomplexität.      |
| Mehr Neuronen = bessere Annäherung | Stimmt, aber mit abnehmendem Grenznutzen.                  |
| Praktisch: mehrere Hidden Layer    | Effizienter und genauer als ein sehr breiter Single-Layer. |

### Fazit für die Neuronenzahl

Das Universal Approximation Theorem motiviert die Wahl einer hinreichend großen Neuronenzahl – es zeigt, dass damit theoretisch alles möglich ist. Es sagt aber nicht, wie viele Neuronen für ein konkretes Problem ausreichend sind. In der Praxis: lieber kleineres Netz mit guter Generalisierung, als ein riesiges.

## Übungsskript für Neuronenzahl

```

import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np

# Ziel-Funktion
def target_function(x):
    return np.sin(2 * np.pi * x) + 0.5 * np.cos(6 * np.pi * x)
X = np.linspace(0, 1, 500).reshape(-1, 1).astype(np.float32)
y = target_function(X).astype(np.float32)

# In Torch-Tensoren umwandeln
X_tensor = torch.from_numpy(X)
y_tensor = torch.from_numpy(y).squeeze()

# Zwei einfache Netzwerke
net1 = nn.Sequential(
    nn.Linear(1, 3),
    nn.Tanh(),
    nn.Linear(3, 1)
)

net2 = nn.Sequential(
    nn.Linear(1, 40),
    nn.Tanh(),
    nn.Linear(40, 1)
)

# Trainingsfunktion mit einfachem Loop
def train(model, X, y, epochs=5000, lr=0.1):
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    for _ in range(epochs):
        optimizer.zero_grad()
        output = model(X).squeeze()
        loss = criterion(output, y)
        loss.backward()
        optimizer.step()

# Modelle trainieren
train(net1, X_tensor, y_tensor)
train(net2, X_tensor, y_tensor)

# Vorhersagen berechnen
with torch.no_grad():
    y_pred1 = net1(X_tensor).squeeze().numpy()
    y_pred2 = net2(X_tensor).squeeze().numpy()

# Plotten
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(X, y, label="Ziel-Funktion", linewidth=2)
plt.plot(X, y_pred1, label="Netz mit 3 Neuronen", linestyle="--")
plt.title("net1: 1 Hidden Layer, 3 Neuronen")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(X, y, label="Ziel-Funktion", linewidth=2)
plt.plot(X, y_pred2, label="Netz mit 40 Neuronen", linestyle="--")
plt.title("net2: 1 Hidden Layer, 40 Neuronen")
plt.legend()

plt.show()

```

## Rang-Reihenfolge: Training nach Wichtigkeit

### 1. Daten (Aufbereitung, Qualität, Menge & DataLoader)

Absolut fundamental. Ohne Daten gibt es kein Lernen. Die Qualität, Repräsentativität und Menge der Daten bestimmen maßgeblich die Obergrenze der möglichen Modell-Performance.

### 2. Modelldefinition (Architektur)

Absolut fundamental. Die Architektur muss zum Problem passen. Eine ungeeignete Architektur wird keine guten Ergebnisse liefern.

### 3. Verlustfunktion (Loss Function)

Absolut fundamental. Definiert das Ziel des Trainings. Eine falsche Loss-Funktion führt dazu, dass das Netz etwas Falsches lernt.

### 4. Optimierer & Lernrate

Absolut fundamental. Der Optimizer aktualisiert die Gewichte basierend auf dem Gradienten. Die Lernrate ist der wichtigste Hyperparameter – zu hoch führt zu Divergenz, zu niedrig zu extrem langsamem Training.

### 5. Aktivierungsfunktionen

Sehr fundamental. Sie bringen die Nichtlinearität ins Netz, ohne die ein tiefes Netz nur eine lineare Transformation wäre.

### 6. Trainings- & Validierungs-Loop

Fundamental für die Durchführung. Die korrekte Implementierung stellt sicher, dass Daten gesehen, Fehler berechnet, Gradienten berechnet und Gewichte aktualisiert werden.

### 7. Batch Size

Sehr hoher Einfluss. Beeinflusst die Stabilität des Gradienten, die Trainingsgeschwindigkeit und die Generalisierungsfähigkeit.

### 8. Gewichtsinitialisierung

Hoher Einfluss, v. a. zu Beginn. Schlechte Initialisierung kann das Training massiv verlangsamen oder verhindern (Vanishing/Exploding Gradients).

### 9. Regularisierungstechniken (Dropout, Weight Decay)

Wichtig für Generalisierung. Verhindern oder reduzieren Überanpassung (Overfitting).

### 10. Normalisierungsschichten (z. B. Batch Normalization)

Sehr wichtig für Stabilität und Geschwindigkeit bei tiefen Netzen.

### 11. Early Stopping

Sehr wichtig für Effizienz und Generalisierung. Stoppt, wenn sich die Leistung auf den Validierungsdaten nicht mehr verbessert.

### 12. Datenaugmentation

Wichtig für Robustheit und Generalisierung, v. a. bei Bilddaten.

**Zusammenfassend: Die ersten 5–6 Punkte sind absolut essenziell, um überhaupt ein Training starten zu können. Die nachfolgenden Punkte sind entscheidend, um das Training stabil, effizient und erfolgreich zu machen und eine gute Generalisierung auf neuen Daten zu erreichen.**

## Simple-Skript mit Batch

```
import torch
import torch.nn as nn
import torch.optim as optim
# Import zusätzlich benötigter Klassen
from torch.utils.data import TensorDataset, DataLoader
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import numpy as np

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

X_train = torch.from_numpy(X_train).float()
X_test = torch.from_numpy(X_test).float()
y_train = torch.from_numpy(y_train).long()
y_test = torch.from_numpy(y_test).long()

# NEU: Erstellen eines TensorDatasets und DataLoaders
train_dataset = TensorDataset(X_train, y_train)
batch_size = 12
# shuffle=True sorgt dafür, dass die Daten in jeder Epoche gemischt werden
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

net = nn.Sequential(
    nn.Linear(4, 10),
    nn.ReLU(),
    nn.Linear(10, 3)
)

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(net.parameters(), lr=0.01)

# Iteration über den DataLoader statt direkt über X_train/y_train
num_epochs = 100
for epoch in range(num_epochs):
    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()
        outputs = net(batch_X)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()
```

## Zusammenfassung Batch-Skript

**Zusätzliche Imports:** TensorDataset und DataLoader wurden aus torch.utils.data importiert.

**Erstellung von TensorDataset:** Die Trainingstensoren (X\_train, y\_train) wurden in ein TensorDataset-Objekt gepackt.

**Erstellung von DataLoader:** Ein DataLoader wurde mit batch\_size=12 und shuffle=True erstellt. shuffle=True ist wichtig für das Training, um sicherzustellen, dass das Modell nicht die Reihenfolge der Daten lernt.

**Angepasste Trainingsschleife:** Die innere Logik wurde durch eine Schleife ersetzt, die über den train\_loader iteriert. In jeder Iteration werden ein Batch von Features (batch\_X) und die zugehörigen Labels (batch\_y) geladen.

## Variablenübersicht (Dimensionalität)

| Bezeichnung       | Dimensionalität | Verwendung                               |
|-------------------|-----------------|------------------------------------------|
| X_train           | (120, 4)        | Trainingsmerkmale (Tensor)               |
| y_train           | (120,)          | Trainingslabels (Tensor, Klassenindizes) |
| X_test            | (30, 4)         | Testmerkmale (Tensor)                    |
| y_test            | (30,)           | Testlabels (Tensor, Klassenindizes)      |
| batch_size        | Skalar (12)     | Konfiguration DataLoader                 |
| lr                | Skalar (0.01)   | Konfiguration Optimizer                  |
| num_epochs        | Skalar (100)    | Anzahl Trainingsdurchläufe               |
| batch_X           | (12, 4)         | Merkmale eines Batches                   |
| batch_y           | (12,)           | Labels eines Batches                     |
| outputs           | (12, 3)         | Modellausgabe (Logits) pro Batch         |
| loss              | Skalar ()       | Berechneter Verlust pro Batch            |
| Gewichte, Layer 1 | (10, 4)         | Trainierbare Modellparameter             |
| Biases, Layer 1   | (10,)           | Trainierbare Modellparameter             |
| Gewichte, Layer 2 | (3, 10)         | Trainierbare Modellparameter             |
| Biases, Layer 2   | (3,)            | Trainierbare Modellparameter             |

## Interpretation: Erstes und zweites Moment beim Optimizer

Stell dir vor, du stehst auf einem hügeligen Gelände – das ist deine Verlustfunktion – und du möchtest den Weg ins tiefste Tal finden. Der Gradient zeigt an deinem aktuellen Punkt immer in die Richtung des steilsten Anstiegs. Um nach unten zu kommen, gehst du in die entgegengesetzte Richtung.

Da du beim Training mit neuronalen Netzen immer nur kleine Ausschnitte der Daten (Mini-Batches) siehst, ist der berechnete Gradient oft etwas verrauscht – er zeigt nicht immer exakt in die beste Richtung. Hier kommen die Momente ins Spiel:

### 1. Erstes Moment (des Gradienten): Der „durchschnittliche Schwung“

Ein gleitender Durchschnitt der Gradienten über die letzten Schritte. Es merkt sich, in welche Richtung die Gradienten im Mittel gezeigt haben. Anschaulich: Das erste Moment ist wie dein gesammelter Schwung oder die Hauptrichtung, in die du dich zuletzt bewegt hast. Es glättet die Richtung und beschleunigt das Vorankommen in konstanten Richtungen. Kleine Richtungsänderungen oder Rauschen werden abgeschwächt.

### 2. Zweites Moment (des Gradienten): Die „durchschnittliche Stärke“

Ein gleitender Durchschnitt der quadrierten Gradienten. Durch das Quadrieren werden alle Werte positiv und größere Gradienten stärker gewichtet. Das zweite Moment misst, wie steil das Gelände zuletzt war – unabhängig von der Richtung. Der Nutzen im Optimizer (z. B. RMSprop, Adam): Es passt die Schrittgröße (Lernrate) automatisch an – bei hoher durchschnittlicher Steilheit kleinere Schritte, bei geringer Steilheit größere Schritte.

## Variante: Simple-Skript mit Early Stopping

```

import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.25,
random_state=42)

X_train = torch.from_numpy(X_train).float()
X_val = torch.from_numpy(X_val).float()
X_test = torch.from_numpy(X_test).float()
y_train = torch.from_numpy(y_train).long()
y_val = torch.from_numpy(y_val).long()
y_test = torch.from_numpy(y_test).long()

net = nn.Sequential(
    nn.Linear(4, 10),
    nn.ReLU(),
    nn.Linear(10, 3)
)

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(net.parameters(), lr=0.01)

batch_size = 16
train_dataset = torch.utils.data.TensorDataset(X_train, y_train)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size,
shuffle=True)

# Early Stopping-Parameter
patience = 10
min_delta = 0.001
best_loss = float('inf')
counter = 0

# Trainieren des neuronalen Netzes
for epoch in range(100):
    net.train()
    for i, data in enumerate(train_loader):
        inputs, labels = data
        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

# Auswerten auf den Validierungsdaten
net.eval()
with torch.no_grad():
    outputs = net(X_val)
    val_loss = criterion(outputs, y_val)
    _, predicted = torch.max(outputs, 1)
    accuracy = accuracy_score(y_val, predicted)

```

```
# Early Stopping-Logik
if val_loss < best_loss - min_delta:
    best_loss = val_loss
    counter = 0
    # Bestes Modell speichern (nur state_dict)
    torch.save(net.state_dict(), 'best_iris_net.pth')
else:
    counter += 1
    if counter >= patience:
        print('Early Stopping: Training abgebrochen')
        break

# Bestes Modell laden
net = nn.Sequential(
    nn.Linear(4, 10),
    nn.ReLU(),
    nn.Linear(10, 3)
)
net.load_state_dict(torch.load('best_iris_net.pth'))
net.eval()

with torch.no_grad():
    outputs = net(X_test)
    _, predicted = torch.max(outputs, 1)
    accuracy = accuracy_score(y_test, predicted)
    print('Testgenauigkeit: ', accuracy)

# Abspeichern des trainierten Netzes
torch.save(net.state_dict(), 'iris_net_early_stopping.pth')
print("Trainiertes Netz abgespeichert als 'iris_net_early_stopping.pth'")
```



## Variante: Simple-Skript mit StandardScaler

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
import joblib

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Normalisierung mit StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.from_numpy(X_train).float()
X_test = torch.from_numpy(X_test).float()
y_train = torch.from_numpy(y_train).long()
y_test = torch.from_numpy(y_test).long()

net = nn.Sequential(
    nn.Linear(4, 10),
    nn.ReLU(),
    nn.Linear(10, 3)
)

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(net.parameters(), lr=0.01)

for epoch in range(100):
    optimizer.zero_grad()
    outputs = net(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

net.eval()
with torch.no_grad():
    outputs = net(X_test)
    _, predicted = torch.max(outputs, 1)
    accuracy = accuracy_score(y_test, predicted)
    print(f'Testgenauigkeit: {accuracy:.4f}')
```

```
# Abspeichern des trainierten Netzes und des Scalers
torch.save(net.state_dict(), 'iris_net_scaled.pth')
joblib.dump(scaler, 'scaler.pkl')
print("Trainiertes Netz abgespeichert als 'iris_net_scaled.pth'")
print("Scaler abgespeichert als 'scaler.pkl'")
```

## Zugehöriges Inference-Skript mit Scaler

```
import torch
import torch.nn.functional as F
import torch.nn as nn
import joblib

# Laden des trainierten Netzes und des StandardScalers
net = nn.Sequential(
    nn.Linear(4, 10),
    nn.ReLU(),
    nn.Linear(10, 3)
)
net.load_state_dict(torch.load('iris_net_scaled.pth'))
scaler = joblib.load('scaler.pkl')
net.eval()

# Eingabe der vier Merkmale
print("Geben Sie die vier Merkmale ein:")
s_length = float(input("Sepal-Länge (cm): "))
s_width = float(input("Sepal-Breite (cm): "))
p_length = float(input("Petal-Länge (cm): "))
p_width = float(input("Petal-Breite (cm): "))

# Skalierung der Eingabewerte
inputs_scaled = scaler.transform([[s_length, s_width, p_length, p_width]])

# Erstellen eines Tensors aus den skalierten Eingabewerten
inputs = torch.tensor(inputs_scaled, dtype=torch.float32)

# Vorhersage treffen
with torch.no_grad():
    outputs = net(inputs)
    _, predicted = torch.max(outputs, 1)

# Ausgabe der Klassifizierungsaussage
iris_classes = ['Setosa', 'Versicolor', 'Virginica']
print("Die vorhergesagte Klasse ist: ", iris_classes[predicted.item()])

# Klassenwahrscheinlichkeiten berechnen
probabilities = F.softmax(outputs, dim=1)
print("Klassenwahrscheinlichkeiten:", probabilities.numpy()[0])
```

## Änderungen zum Original-Skript

**StandardScaler** wurde hinzugefügt, um die Eingabewerte zu normalisieren. Die normalisierten Daten werden erst nach der Skalierung in Tensoren umgewandelt. Der StandardScaler wird nicht automatisch im Modell mit abgespeichert, daher muss dieser separat gespeichert und beim Laden des Modells wieder geladen und auf die Eingabedaten angewandt werden.

## Simple Visualisierung

### Variante 1

```
import matplotlib.pyplot as plt

fig, axs = plt.subplots(2, 1, figsize=(8, 10))

net.eval()
with torch.no_grad():
    y_pred = net(X_train)          # KORREKTUR: war 'model1' → 'net'
    predicted = torch.argmax(y_pred, dim=1)

# Oben: Originaldaten
axs[0].set_title("Originaldaten")
axs[0].scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='plasma')

# Unten: Vorhersagen
axs[1].set_title("Vorhersagen")
axs[1].scatter(X_train[:, 0], X_train[:, 1], c=predicted, cmap='plasma')

plt.tight_layout()
plt.show()
```

### Variante 2

```
import matplotlib.pyplot as plt

net.eval()
with torch.no_grad():
    outputs_train = net(X_train)
    _, predicted_train = torch.max(outputs_train, 1)

# Visualisierung mit Matplotlib
plt.figure(figsize=(6, 8))

plt.subplot(2, 1, 1)
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train)
plt.title('Original-Trainingsdaten')
plt.xlabel('Sepal-Länge (cm)')
plt.ylabel('Sepal-Breite (cm)')

plt.subplot(2, 1, 2)
plt.scatter(X_train[:, 0], X_train[:, 1], c=predicted_train)
plt.title('Vorhergesagte Trainingsdaten')
plt.xlabel('Sepal-Länge (cm)')
plt.ylabel('Sepal-Breite (cm)')
```

```
plt.tight_layout()  
plt.show()
```

## Trainings- und Validierungsverlauf plotten (Loss-Kurven)

Um Overfitting frühzeitig zu erkennen, speichern wir in jedem Epoch den Train- und Val-Loss und visualisieren beide Kurven. Steigt der Val-Loss während der Train-Loss weiter fällt, ist das ein klares Zeichen für Overfitting.

```
import torch
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import numpy as np

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2, random_state=0)

X_train = torch.from_numpy(X_train).float()
X_val = torch.from_numpy(X_val).float()
X_test = torch.from_numpy(X_test).float()
y_train = torch.from_numpy(y_train).long()
y_val = torch.from_numpy(y_val).long()
y_test = torch.from_numpy(y_test).long()

net = nn.Sequential(nn.Linear(4, 10), nn.ReLU(), nn.Linear(10, 3))
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(net.parameters(), lr=0.01)

# Listen für die Verlaufskurven
train_losses = []
val_losses = []

for epoch in range(100):
    net.train()
    optimizer.zero_grad()
    outputs = net(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()
    train_losses.append(loss.item())

    net.eval()
    with torch.no_grad():
        val_out = net(X_val)
        val_loss = criterion(val_out, y_val)
    val_losses.append(val_loss.item())

# Verlaufskurven plotten
plt.figure(figsize=(8, 4))
plt.plot(train_losses, label='Train-Loss')
```

```
plt.plot(val_losses, label='Val-Loss')
plt.xlabel('Epoche')
plt.ylabel('Loss')
plt.title('Trainings- und Validierungsverlauf')
plt.legend()
plt.tight_layout()
plt.show()
```

## Confusion Matrix

Die Confusion Matrix zeigt auf einen Blick, welche Klassen gut und welche schlecht klassifiziert werden. Sie ist informativer als die reine Accuracy, weil sie auch Verwechslungsmuster sichtbar macht.

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# net, X_test, y_test müssen aus dem Trainings-Skript vorliegen
net.eval()
with torch.no_grad():
    outputs = net(X_test)
    _, predicted = torch.max(outputs, 1)

iris_classes = ['Setosa', 'Versicolor', 'Virginica']
cm = confusion_matrix(y_test, predicted)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=iris_classes)
disp.plot(cmap='Blues')
plt.title('Confusion Matrix - Iris-Klassifikation')
plt.tight_layout()
plt.show()
```

**Interpretation:** Die Diagonale (oben links nach unten rechts) zeigt korrekte Vorhersagen. Einträge außerhalb der Diagonale sind Fehler. Im Iris-Datensatz werden Setosa (Klasse 0) meist perfekt erkannt, während Versicolor und Virginica (Klassen 1 und 2) gelegentlich verwechselt werden.

## Variante mit Learning Rate Scheduler

Der Learning Rate Scheduler passt die Lernrate während des Trainings automatisch an. Hier nutzen wir ReduceLROnPlateau: wird der Val-Loss für 5 aufeinanderfolgende Epochen nicht besser, wird die Lernrate halbiert. Das stabilisiert das Training in der Feinabstimmungsphase und ergänzt Early Stopping.

```
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import numpy as np

iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.2,
random_state=0)

X_train = torch.from_numpy(X_train).float()
X_val = torch.from_numpy(X_val).float()
X_test = torch.from_numpy(X_test).float()
y_train = torch.from_numpy(y_train).long()
y_val = torch.from_numpy(y_val).long()
y_test = torch.from_numpy(y_test).long()

net = nn.Sequential(
    nn.Linear(4, 10),
    nn.ReLU(),
    nn.Linear(10, 3)
)

criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(net.parameters(), lr=0.01)

# ReduceLROnPlateau: halbiert LR, wenn Val-Loss sich 5 Epochen nicht verbessert
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', factor=0.5, patience=5, verbose=True
)

for epoch in range(100):
    net.train()
    optimizer.zero_grad()
    outputs = net(X_train)
    loss = criterion(outputs, y_train)
    loss.backward()
    optimizer.step()

    net.eval()
```



```
with torch.no_grad():
    val_out = net(X_val)
    val_loss = criterion(val_out, y_val)

    scheduler.step(val_loss)          # LR bei Stagnation reduzieren
    current_lr = optimizer.param_groups[0]['lr']
    print(f'Epoch {epoch:3d} | Train: {loss.item():.4f} | Val: {val_loss.item():.4f} |
    LR: {current_lr:.6f}')
```

## Wichtige Parameter von ReduceLROnPlateau

mode='min' – Überwacht einen Wert, der kleiner werden soll (hier: Loss).

factor=0.5 – Neue LR = alte LR  $\times$  0.5 (Halbierung).

patience=5 – Warte 5 Epochen ohne Verbesserung, bevor die LR gesenkt wird.

verbose=True – Gibt eine Meldung aus, wenn die LR angepasst wird.

## Weitere nützliche Scheduler

StepLR(optimizer, step\_size=30, gamma=0.1) – Senkt die LR alle 30 Epochen um den Faktor 0.1.

CosineAnnealingLR(optimizer, T\_max=100) – Reduziert die LR nach einem Kosinus-Schema bis auf nahe 0.